Foundation v3.1 Addendum — Three Missing Pieces

Transport Layer + Generic Deterministic IDs + Foundation/Utils Separation

> **Version:** 3.1 | **Status:** Addendum to Foundation_Utils_REQ_DES_TASKS_v3_0.md **Date:** 2025-02-25
> **Scope:** 3 structural changes to shared/ crate **Author:** Specification pipeline

---

TABLE OF CONTENTS

PART A: Transport / Communication Layer

A.1 Problem Statement

The current Foundation spec defines **5 cross-system traits** (M7) but has **NO transport mechanism** for how systems actually communicate. The traits define WHAT the API contract is, but not HOW messages get from Praman → Siddhanta or Dhruva → Siddhanta.

Currently missing:

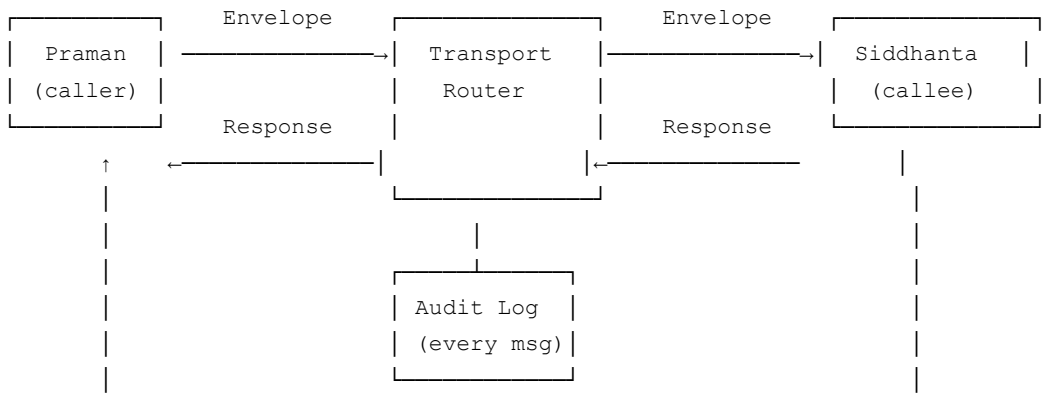
- No message envelope type (who sent, who receives, what payload, which trace)
- No call protocol (sync vs async vs fire-and-forget)
- No deterministic message ordering guarantee
- No error propagation across system boundaries
- No message routing / dispatch
- No message serialization contract for cross-system calls

- No timeout / deadline propagation

A.2 Design Principles

```
RULE 1: ALL cross-system communication goes through Transport.
         No direct function calls between crates. Ever.

RULE 2: Transport is deterministic.
         Same messages in same order → same results.

RULE 3: Transport is local-first.
         All 5 systems run in the same process by default.
         Network transport is a future extension, NOT in v3.1.

RULE 4: Transport carries DeterministicContext metadata.
         Every message has a TraceId and LogicalTimestamp.

RULE 5: Transport is auditable.
         Every cross-system call is logged with full envelope.
```

A.3 Architecture



In-process (v3.1): Transport is a trait-dispatch router. Caller builds Envelope, Router dispatches to callee's trait impl, callee returns Response. All synchronous, all in-process.

Future (v4+): Transport can be swapped for IPC/gRPC/message queue without changing any caller/callee code — only the Router impl changes.

M19: Transport Types

File: `foundation/src/transport.rs`

Requirements

ID	Requirement	Priority	Trace
REQ-M19-01	System SHALL define `Envelope<P>` as the universal cross-system message wrapper	P0	Cross-Call Map
REQ-M19-02	Envelope SHALL contain: source (SystemId), target (SystemId), trace_id (TraceId), timestamp (LogicalTimestamp), payload (P), deadline_ticks (Option<u64>)	P0	Determinism
REQ-M19-03	System SHALL define `Response<R>` with: trace_id, source, result (Result<R, TransportError>), duration_ticks (u64)	P0	Audit
REQ-M19-04	System SHALL define `TransportError` enum covering: Timeout, TargetUnavailable, PayloadTooLarge, SerializationFailed, CalleeError(FoundationError), DeadlineExceeded	P0	Error catalog
REQ-M19-05	Envelope SHALL enforce payload size ≤ MAX_TRANSPORT_PAYLOAD (default 4MB via BoundedBuffer)	P0	V5 resolution
REQ-M19-06	All Envelope fields SHALL use v3.0 types only (TraceId, LogicalTimestamp, SystemId — NO String, NO wall-clock)	P0	v3.0
REQ-M19-07	Envelope and Response SHALL implement DeterministicSerialize + DeterministicDeserialize	P0	M9
REQ-M19-08	System SHALL define `CallMode` enum: Sync, FireAndForget	P0	Protocol

Design

```
// foundation/src/transport.rs

/// Universal cross-system message envelope
pub struct Envelope<P: DeterministicSerialize> {
    pub source:         SystemId,
    pub target:         SystemId,
    pub trace_id:       TraceId,
    pub timestamp:      LogicalTimestamp,
    pub call_mode:      CallMode,
    pub deadline_ticks: Option<u64>,        // None = no deadline
    pub payload:        P,
}

/// Response wrapper
pub struct Response<R: DeterministicSerialize> {
```

```
    pub trace_id:       TraceId,            // echo back for correlation
    pub source:         SystemId,           // who responded
    pub result:         Result<R, TransportError>,
    pub duration_ticks: u64,                // callee-measured processing time
}

/// Call protocol modes
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(u8)]
pub enum CallMode {
    Sync           = 0,   // Wait for Response
    FireAndForget  = 1,   // No response expected (audit-only)
}

/// Transport-level errors
#[derive(Debug, Clone)]
pub enum TransportError {
    Timeout { deadline_ticks: u64, elapsed_ticks: u64 },
    TargetUnavailable { target: SystemId },
    PayloadTooLarge { size: usize, max: usize },
    SerializationFailed { reason: String },
    CalleeError(FoundationError),
    DeadlineExceeded { trace_id: TraceId },
    InvalidRoute { source: SystemId, target: SystemId },
}

pub const MAX_TRANSPORT_PAYLOAD: usize = 4 * 1024 * 1024; // 4 MB
```

Tasks

ID	Task	Estimate	Traces
TASK-M19-01	Define Envelope<P> struct with all 7 fields	1h	REQ-M19-01, M19-02
TASK-M19-02	Define Response<R> struct with 4 fields	0.5h	REQ-M19-03
TASK-M19-03	Define TransportError enum (7 variants)	1h	REQ-M19-04
TASK-M19-04	Define CallMode enum (2 variants, repr(u8))	0.5h	REQ-M19-08
TASK-M19-05	Implement payload size validation in Envelope constructor	1h	REQ-M19-05
TASK-M19-06	Implement DeterministicSerialize for Envelope and Response	2h	REQ-M19-07
TASK-M19-			

07	Test: Envelope roundtrip serialization 1000× identical	1h	REQ-M19-07
TASK-M19-08	Test: payload > MAX rejected at construction	0.5h	REQ-M19-05
Subtotal		**7.5h**	

M20: Transport Router

File: `foundation/src/router.rs`

Requirements

ID	Requirement	Priority	Trace
REQ-M20-01	System SHALL define `Transport` trait: `fn call<P, R>(&self, envelope: Envelope<P>, ctx: &mut DeterministicContext) → Response<R>`	P0	Core
REQ-M20-02	System SHALL define `InProcessRouter` implementing Transport: dispatches to registered trait objects by SystemId	P0	v3.1 in-process
REQ-M20-03	InProcessRouter SHALL store callees in BTreeMap<SystemId, Box<dyn SystemHandler>> (deterministic iteration)	P0	W4
REQ-M20-04	InProcessRouter.call() SHALL create AuditRecord for EVERY cross-system call (envelope + response)	P0	Audit
REQ-M20-05	InProcessRouter.call() SHALL propagate DeterministicContext to callee (advancing trace_id via sub_id)	P0	V7
REQ-M20-06	For FireAndForget mode, Router SHALL still log the call but return Response with Ok(()) immediately	P0	Protocol
REQ-M20-07	Router SHALL validate source → target route is ALLOWED per call direction rules	P0	Cross-Call Map
REQ-M20-08	System SHALL define `SystemHandler` trait: `fn handle(&self, payload: &[u8], ctx: &DeterministicContext) → Result<Vec<u8>, FoundationError>`	P0	Dispatch

Design

```
// foundation/src/router.rs

/// Trait for cross-system dispatch
pub trait Transport: Send + Sync {
    fn call_raw(
        &self,
        envelope: &Envelope<Vec<u8>>,   // serialized payload
        ctx: &mut DeterministicContext,
    ) -> Response<Vec<u8>>;
}

/// Handler that each system registers
pub trait SystemHandler: Send + Sync {
    fn handle(
        &self,
        method: &str,
        payload: &[u8],
        ctx: &DeterministicContext,
    ) -> Result<Vec<u8>, FoundationError>;
}

/// In-process router (v3.1 default)
pub struct InProcessRouter {
    handlers: BTreeMap<SystemId, Box<dyn SystemHandler>>,
    allowed_routes: BTreeSet<(SystemId, SystemId)>,  // (source, target)
    audit_sink: Box<dyn AuditSink>,
}

impl InProcessRouter {
    pub fn new(audit_sink: Box<dyn AuditSink>) -> Self;
    pub fn register(&mut self, system: SystemId, handler: Box<dyn SystemHandler>);
    pub fn set_allowed_routes(&mut self, routes: &[(SystemId, SystemId)]);
}

/// Allowed routes (from Cross-Call Map)
/// Praman → Siddhanta  ✓
/// Praman → Dhruva     ✓
/// Praman → Tejas      ✓
/// Praman → Dristi     ✓
/// Tejas  → Dristi     ✓
/// Dhruva → Siddhanta  ✓
/// ALL OTHERS           ✗

pub trait AuditSink: Send + Sync {
    fn log_call(&self, envelope: &Envelope<Vec<u8>>, response: &Response<Vec<u8>>);
}
```

Tasks

ID	Task	Estimate	Traces
TASK-M20-01	Define Transport trait with call_raw()	1h	REQ-M20-01
TASK-M20-02	Define SystemHandler trait with handle()	1h	REQ-M20-08
TASK-M20-03	Implement InProcessRouter struct with BTreeMap storage	1.5h	REQ-M20-02, M20-03
TASK-M20-04	Implement register() and set_allowed_routes()	1h	REQ-M20-02, M20-07
TASK-M20-05	Implement call_raw() dispatch: validate route → serialize → dispatch → audit → respond	2h	REQ-M20-04, M20-05
TASK-M20-06	Implement FireAndForget handling in call_raw()	1h	REQ-M20-06
TASK-M20-07	Implement route validation (reject disallowed source→target)	1h	REQ-M20-07
TASK-M20-08	Define AuditSink trait + InMemoryAuditSink (for testing)	1h	REQ-M20-04
TASK-M20-09	Test: Praman→Siddhanta dispatch end-to-end	1.5h	All
TASK-M20-10	Test: Disallowed route (Siddhanta→Praman) rejected	1h	REQ-M20-07
TASK-M20-11	Test: Every call produces exactly 1 AuditRecord	1h	REQ-M20-04
TASK-M20-12	Test: DeterministicContext.sub_id() used for callee trace_id	1h	REQ-M20-05
Subtotal		**14h**	

M21: Typed Cross-System Client

File: `foundation/src/client.rs`

Requirements

ID	Requirement	Priority	Trace
REQ-	System SHALL provide typed client wrappers for each cross-system		

M21-01	boundary (SiddhantaClient, DhruvaClient, TejasClient, DristiClient)	P0	Ergonomics
REQ-M21-02	Each client SHALL wrap Transport, building Envelope and deserializing Response automatically	P0	DRY
REQ-M21-03	Client methods SHALL match the M7 trait signatures exactly (same names, same types)	P0	Consistency
REQ-M21-04	Client SHALL automatically set source SystemId, target SystemId, and derive trace_id from ctx	P0	Determinism
REQ-M21-05	Client SHALL convert TransportError into system-appropriate FoundationError variants	P0	Error handling

Design

```
// foundation/src/client.rs

/// Typed client for Praman → Siddhanta calls
pub struct SiddhantaClient<'t> {
    transport: &'t dyn Transport,
}

impl<'t> SiddhantaClient<'t> {
    pub fn new(transport: &'t dyn Transport) -> Self;

    pub fn sign(
        &self,
        hash: &Hash256,
        ctx: &mut DeterministicContext,
    ) -> Result<ExecutionCertificate, FoundationError>;

    pub fn verify(
        &self,
        cert: &ExecutionCertificate,
        ctx: &mut DeterministicContext,
    ) -> Result<bool, FoundationError>;

    pub fn append_ledger(
        &self,
        cert: ExecutionCertificate,
        ctx: &mut DeterministicContext,
    ) -> Result<LedgerEntry, FoundationError>;
}

/// Typed client for Praman → Dhruva calls
```

```
pub struct DhruvaClient<'t> { transport: &'t dyn Transport }
/// Typed client for Praman → Tejas calls
pub struct TejasClient<'t> { transport: &'t dyn Transport }
/// Typed client for Praman → Dristi calls
pub struct DristiClient<'t> { transport: &'t dyn Transport }
/// Typed client for Tejas → Dristi calls
pub struct DristiCompilerClient<'t> { transport: &'t dyn Transport }
/// Typed client for Dhruva → Siddhanta calls
pub struct SiddhantaStateClient<'t> { transport: &'t dyn Transport }
```

Tasks

ID	Task	Estimate	Traces
TASK-M21-01	Implement SiddhantaClient (3 methods: sign, verify, append_ledger)	2h	REQ-M21-01..05
TASK-M21-02	Implement DhruvaClient (7 methods from Cross-Call Map §1.2)	2.5h	REQ-M21-01..05
TASK-M21-03	Implement TejasClient (5 methods from Cross-Call Map §1.3)	2h	REQ-M21-01..05
TASK-M21-04	Implement DristiClient (5 methods from Cross-Call Map §1.4)	2h	REQ-M21-01..05
TASK-M21-05	Implement DristiCompilerClient (3 methods from Cross-Call Map §1.5)	1.5h	REQ-M21-01..05
TASK-M21-06	Implement SiddhantaStateClient (3 methods from Cross-Call Map §1.6)	1.5h	REQ-M21-01..05
TASK-M21-07	Test: Each client method builds correct Envelope (source, target, trace)	2h	REQ-M21-04
TASK-M21-08	Test: TransportError → FoundationError conversion	1h	REQ-M21-05
Subtotal		**14.5h**	

Transport Layer Totals

Module	REQs	Tasks	Hours
M19: Transport Types	8	8	7.5h
M20: Transport Router	8	12	14h
M21: Typed Cross-System Client	5	8	14.5h

Total Part A	**21**	**28**	**36h**

PART B: Generic Deterministic ID System

B.1 Problem Statement

Current spec has only ONE ID type — `TraceId([u8; 16])` generated by `DeterministicContext.next_trace_id()`. But the system needs MANY distinct ID types:

- **TraceId** — pipeline execution traces (Praman)
- **SnapshotId** — state snapshots (Dhruva)
- **LedgerEntryId** — ledger entries (Siddhanta)
- **CertificateId** — execution certificates (Siddhanta)
- **ArtifactId** — rendered artifacts (Dristi)
- **FrameId** — render frames (Dristi)
- **NodeId** — DAG nodes (Tejas)
- **RuleId** — symbolic rules (Praman)
- **SessionId** — user sessions

These were all `String` or `u32` in v1.0/v2.0. In v3.0 we made TraceId correct, but the others are still ad-hoc.

B.2 Design: DeterministicId<TAG>

Core idea: A single generic ID type parameterized by a zero-sized tag type. ALL IDs share the same generation mechanism (SHA-256 from DeterministicContext), but are **type-incompatible** at compile time.

```
DeterministicId<TraceTag>    cannot be used where DeterministicId<SnapshotTag> is expected.
                              ^^^ compile error
```

This eliminates the class of bugs where a SnapshotId is accidentally passed as a CertificateId.

M22: Generic Deterministic ID Framework

File: `foundation/src/id.rs`

Requirements

ID	Requirement	Priority	Trace
REQ-M22-01	System SHALL define `DeterministicId<T: IdTag>` as a generic newtype wrapping `[u8; 16]`	P0	Type safety
REQ-M22-02	`IdTag` SHALL be a sealed marker trait with `const DOMAIN: &'static [u8]` — used as salt in SHA-256 derivation	P0	Domain separation
REQ-M22-03	DeterministicId generation SHALL be: `SHA-256(tag.DOMAIN ‖ session_seed ‖ logical_time ‖ counter)[0..16]`	P0	Determinism + timestamp-embedded
REQ-M22-04	DeterministicContext SHALL provide `fn next_id<T: IdTag>(&mut self) → DeterministicId<T>` as the ONLY generation path	P0	V3 resolution
REQ-M22-05	DeterministicId SHALL encode LogicalTimestamp in bytes [0..8] and uniqueness in bytes [8..16] for natural ordering	P0	Timestamp-embedded
REQ-M22-06	DeterministicId SHALL implement Ord using byte comparison (which gives temporal ordering due to big-endian timestamp)	P0	Sortable by time
REQ-M22-07	System SHALL define tag types: TraceTag, SnapshotTag, LedgerTag, CertificateTag, ArtifactTag, FrameTag, NodeTag, RuleTag, SessionTag	P0	All ID domains
REQ-M22-08	Type aliases SHALL be defined: `type TraceId = DeterministicId<TraceTag>` , etc.	P0	Ergonomics
REQ-M22-09	DeterministicId SHALL implement Display as `"{domain}-{hex}"` (e.g., `"trace-a1b2c3d4..."`)	P0	Human-readable
REQ-M22-10	DeterministicId<A> SHALL NOT be convertible to DeterministicId<B> (no From/Into across tags)	P0	Type safety
REQ-M22-11	DeterministicId SHALL implement DeterministicSerialize, serializing as exactly 16 bytes in binary and 36-char string in JSON	P0	M9
REQ-M22-	DeterministicId SHALL NOT implement Default (no uninitialized IDs)	P0	Safety

12			

Design

```
// foundation/src/id.rs

/// Sealed marker trait for ID domain separation
pub trait IdTag: 'static {
    /// Domain salt bytes — used in SHA-256 derivation to prevent cross-domain collisions
    const DOMAIN: &'static [u8];
    /// Human-readable prefix for Display
    const PREFIX: &'static str;
}

/// Generic deterministic ID — type-safe, timestamp-embedded, deterministic
/// Layout:
///   bytes [0..8]:  LogicalTimestamp (big-endian u64) — enables temporal ordering
///   bytes [8..16]: Uniqueness hash (from SHA-256 derivation)
#[derive(Clone, Copy, PartialEq, Eq, Hash)]
pub struct DeterministicId<T: IdTag> {
    bytes: [u8; 16],
    _tag: PhantomData<T>,
}

impl<T: IdTag> DeterministicId<T> {
    /// Extract the embedded LogicalTimestamp
    pub fn timestamp(&self) -> LogicalTimestamp {
        LogicalTimestamp(u64::from_be_bytes(self.bytes[0..8].try_into().unwrap()))
    }

    /// Display as hex: "trace-a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5"
    pub fn to_hex(&self) -> String;

    /// Parse from hex: "trace-a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5"
    pub fn from_hex(hex: &str) -> Result<Self, FoundationError>;
}

// Ord uses byte comparison — big-endian timestamp in [0..8] gives temporal ordering
impl<T: IdTag> Ord for DeterministicId<T> { ... }

// === Tag types (each is a zero-sized struct) ===

pub struct TraceTag;
impl IdTag for TraceTag {
    const DOMAIN: &'static [u8] = b"trace";
    const PREFIX: &'static str = "trace";
}

pub struct SnapshotTag;
impl IdTag for SnapshotTag {
    const DOMAIN: &'static [u8] = b"snapshot";
```

```
    const PREFIX: &'static str = "snap";
}

pub struct LedgerTag;
impl IdTag for LedgerTag {
    const DOMAIN: &'static [u8] = b"ledger";
    const PREFIX: &'static str = "ledg";
}

pub struct CertificateTag;
impl IdTag for CertificateTag {
    const DOMAIN: &'static [u8] = b"certificate";
    const PREFIX: &'static str = "cert";
}

pub struct ArtifactTag;
impl IdTag for ArtifactTag {
    const DOMAIN: &'static [u8] = b"artifact";
    const PREFIX: &'static str = "artf";
}

pub struct FrameTag;      // PREFIX: "frm"
pub struct NodeTag;       // PREFIX: "node"
pub struct RuleTag;       // PREFIX: "rule"
pub struct SessionTag;    // PREFIX: "sess"

// === Type aliases (what all systems actually use) ===

pub type TraceId       = DeterministicId<TraceTag>;
pub type SnapshotId    = DeterministicId<SnapshotTag>;
pub type LedgerEntryId = DeterministicId<LedgerTag>;
pub type CertificateId = DeterministicId<CertificateTag>;
pub type ArtifactId    = DeterministicId<ArtifactTag>;
pub type FrameId       = DeterministicId<FrameTag>;
pub type NodeId        = DeterministicId<NodeTag>;
pub type RuleId        = DeterministicId<RuleTag>;
pub type SessionId     = DeterministicId<SessionTag>;
```

Generation in DeterministicContext (updated M1):

```
impl DeterministicContext {
    /// Generate any typed ID — the ONLY way to create DeterministicId
    pub fn next_id<T: IdTag>(&mut self) -> DeterministicId<T> {
        self.id_counter += 1;
        let hash = sha256(&[
            T::DOMAIN,
            &self.session_seed,
            &self.logical_clock.0.to_be_bytes(),
            &self.id_counter.to_be_bytes(),
        ].concat());

        let mut bytes = [0u8; 16];
```

```
        // Bytes [0..8]: LogicalTimestamp (big-endian) for temporal ordering
        bytes[0..8].copy_from_slice(&self.logical_clock.0.to_be_bytes());
        // Bytes [8..16]: SHA-256 hash truncation for uniqueness
        bytes[8..16].copy_from_slice(&hash[0..8]);

        DeterministicId { bytes, _tag: PhantomData }
    }

    // Legacy compatibility — calls next_id<TraceTag>()
    pub fn next_trace_id(&mut self) -> TraceId {
        self.next_id::<TraceTag>()
    }
}
```

Tasks

ID	Task	Estimate	Traces
TASK-M22-01	Define IdTag sealed trait with DOMAIN and PREFIX constants	1h	REQ-M22-02
TASK-M22-02	Define DeterministicId<T> struct with [u8;16] + PhantomData	1h	REQ-M22-01
TASK-M22-03	Implement timestamp() extraction (big-endian u64 from bytes[0..8])	0.5h	REQ-M22-05
TASK-M22-04	Implement Ord via byte comparison, verify it gives temporal ordering	1h	REQ-M22-06
TASK-M22-05	Implement Display as “{prefix}-{hex}” and from_hex() parser	1.5h	REQ-M22-09
TASK-M22-06	Define all 9 tag types (TraceTag through SessionTag)	1h	REQ-M22-07
TASK-M22-07	Define all 9 type aliases (TraceId through SessionId)	0.5h	REQ-M22-08
TASK-M22-08	Implement DeterministicSerialize / DeterministicDeserialize	1.5h	REQ-M22-11
TASK-M22-09	Update DeterministicContext: add id_counter field, implement next_id<T>()	2h	REQ-M22-03, M22-04
TASK-M22-10	Implement next_trace_id() as alias for next_id::<TraceTag>()	0.5h	REQ-M22-04
TASK-M22-11	Test: DeterministicId<TraceTag> cannot be assigned to DeterministicId<SnapshotTag> (compile-fail test)	1h	REQ-M22-10

TASK-M22-12	Test: IDs generated in sequence have increasing timestamps (temporal ordering)	1h	REQ-M22-05, M22-06
TASK-M22-13	Test: Same seed + same sequence → same IDs (determinism) 1000×	1h	REQ-M22-03
TASK-M22-14	Test: Different tags with same context → different IDs (domain separation)	0.5h	REQ-M22-02
TASK-M22-15	Test: Display roundtrip (to_hex → from_hex) for all 9 types	1h	REQ-M22-09
TASK-M22-16	Test: No Default impl exists (compile-fail test)	0.5h	REQ-M22-12
TASK-M22-17	Verify: no bare [u8;16] or u64 used as ID anywhere in compound types	1h	v3.1 cleanup
Subtotal		**16h**	

Updated M1 (DeterministicContext) — Changes Only

ID	Change	Traces
REQ-M1-04-v3.1	next_trace_id() becomes alias for next_id::<TraceTag>()	Backward compat
REQ-M1-13	DeterministicContext SHALL have id_counter: u64 field (replaces trace_counter)	REQ-M22-04
REQ-M1-14	DeterministicContext.next_id<T: IdTag>() SHALL be the SOLE factory for ALL DeterministicId types	REQ-M22-04
REQ-M1-15	snapshot() SHALL capture id_counter (in addition to existing fields)	Replay

No new tasks needed — TASK-M22-09/M22-10 already cover the DeterministicContext updates.

Updated Compound Types — Changes

These compound types from M3-M6 must update their ID fields:

Struct	Old Field	New Field
StateFingerprint	—	snapshot_id: SnapshotId
ExecutionCertificate	—	certificate_id: CertificateId
LedgerEntry	—	entry_id: LedgerEntryId

RenderedArtifact	—	artifact_id: ArtifactId
ExecutionGraph	node IDs: u32	node IDs: NodeId
DecisionOutput	—	trace_id: TraceId (already)
AuditRecord	—	trace_id: TraceId (already)

Additional task:

ID	Task	Estimate	Traces
TASK-M22-18	Update M3-M6 compound types to use typed DeterministicId	3h	All compound types

Part B Total: 12 REQs, 18 tasks, 19h

PART C: Foundation vs Utils Crate Separation

C.1 Problem Statement

Current `shared/` crate bundles EVERYTHING — types, context, traits, hashing, serialization, math, validation, graph algorithms. This creates problems:

1. **Dristi doesn't need nalgebra** — but it's compiled because shared/ depends on it
2. **Siddhanta doesn't need graph helpers** — but they're in the same crate
3. **Compile times** — changing a math helper recompiles ALL systems
4. **Conceptual clarity** — "foundation" (types, context, crypto) is different from "utilities" (math, graph, linalg)

C.2 Split Design

```
BEFORE (v3.0):
shared/
├── everything mixed together

AFTER (v3.1):
foundation/          ← types, context, traits, transport, IDs, crypto, errors
│                      EVERY system depends on this. Minimal dependencies.
│
utils/               ← math, linalg, graph, validation, audit, determinism helpers
│                      Systems depend on what they need. Optional dependencies.
│
foundation depends on: sha2, ed25519-dalek, libm, serde
```

```
utils depends on:      foundation, nalgebra (optional), libm
```

C.3 Module Assignment

`foundation/` crate — What EVERY system needs

Module	File	Reason
M1: DeterministicContext	foundation/src/context.rs	Every system receives &ctx
M2: Primitive Types	foundation/src/primitives.rs	Every system uses these
M3: Compound Types (Cross-System)	foundation/src/types/cross_system.rs	Shared structs
M7: Cross-System Traits	foundation/src/traits.rs	Interface contracts
M8: Hash Helpers	foundation/src/helpers/hash.rs	SHA-256 used by ALL
M9: Serialization Helpers	foundation/src/helpers/serialize.rs	Canonical JSON used by ALL
M10: Time Helpers	foundation/src/helpers/time.rs	Logical time used by ALL
M15: Audit Helpers	foundation/src/helpers/audit.rs	Audit used by ALL
M18: Compiler & Build Config	foundation/.cargo/config.toml etc.	ALL code depends on FMA flag
M19: Transport Types	foundation/src/transport.rs	Cross-system comms
M20: Transport Router	foundation/src/router.rs	Cross-system comms
M21: Typed Clients	foundation/src/client.rs	Cross-system comms
M22: Generic ID Framework	foundation/src/id.rs	ALL systems generate IDs

`utils/` crate — What SPECIFIC systems need

Module	File	Used By	Feature Gate
M4: Compound Types (Perception)	utils/src/types/perception.rs	Praman	`perception`
M5: Compound Types (Reasoning)	utils/src/types/reasoning.rs	Praman	`reasoning`
M6: Compound Types (Infrastructure)	utils/src/types/infrastructure.rs	Praman, Dhruva	`infra`

M11: Math Helpers	utils/src/helpers/math.rs	Praman, Dristi	`math`
M12: Linear Algebra Helpers	utils/src/helpers/linalg.rs	Praman	`linalg`
M13: Validation Helpers	utils/src/helpers/validate.rs	Praman, Dristi, Dhruva	`validate`
M14: Determinism Helpers	utils/src/helpers/determinism.rs	ALL (but only in test/CI)	`determinism`
M16: ID Helpers	utils/src/helpers/id.rs	Display/parse	`id-display`
M17: Graph Helpers	utils/src/helpers/graph.rs	Tejas, Praman, Dristi	`graph`

Cargo.toml Examples

```
# foundation/Cargo.toml
[package]
name = "tejas-foundation"
version = "3.1.0"

[dependencies]
sha2 = "0.10"
ed25519-dalek = "2.0"
libm = "0.2"
serde = { version = "1", features = ["derive"] }

# NO nalgebra — foundation stays lightweight


# utils/Cargo.toml
[package]
name = "tejas-utils"
version = "3.1.0"

[dependencies]
tejas-foundation = { path = "../foundation" }

[dependencies.nalgebra]
version = "0.32"
default-features = false   # NO BLAS
optional = true

[features]
default = ["math", "validate", "graph", "id-display"]
perception = []
reasoning = ["nalgebra"]
infra = []
math = []
linalg = ["nalgebra"]
```

```
validate = []
determinism = []
id-display = []
graph = []
full = ["perception", "reasoning", "infra", "math", "linalg", "validate", "determinism", "id


# System-specific Cargo.toml examples:

# praman/Cargo.toml
[dependencies]
tejas-foundation = { path = "../foundation" }
tejas-utils = { path = "../utils", features = ["full"] }

# siddhanta/Cargo.toml
[dependencies]
tejas-foundation = { path = "../foundation" }
tejas-utils = { path = "../utils", features = ["validate"] }
# NO nalgebra compiled for Siddhanta!

# dristi/Cargo.toml
[dependencies]
tejas-foundation = { path = "../foundation" }
tejas-utils = { path = "../utils", features = ["math", "validate", "graph"] }
# nalgebra NOT needed for Dristi — no linalg feature
```

Requirements for Separation

ID	Requirement	Priority	Trace
REQ-SPLIT-01	foundation/ SHALL compile with ZERO optional dependencies beyond sha2, ed25519-dalek, libm, serde	P0	Minimal
REQ-SPLIT-02	utils/ SHALL depend on foundation/ and NOTHING else mandatory	P0	Layering
REQ-SPLIT-03	nalgebra SHALL be feature-gated behind `linalg` and `reasoning` features in utils/	P0	Compile time
REQ-SPLIT-04	Every system SHALL depend on foundation/ directly	P0	Universal
REQ-SPLIT-05	Systems SHALL depend on utils/ with ONLY the features they need	P0	Minimal
REQ-			

SPLIT-06	foundation/ SHALL NOT depend on utils/ (no circular dependency)	P0	Layering
REQ-SPLIT-07	CI SHALL verify foundation/ compiles standalone with `cargo build -p tejas-foundation`	P0	Independence
REQ-SPLIT-08	CI SHALL verify utils/ compiles with each feature individually	P0	Feature gates

Tasks

ID	Task	Estimate	Traces
TASK-SPLIT-01	Create foundation/ crate with Cargo.toml (sha2, ed25519-dalek, libm, serde)	1h	REQ-SPLIT-01
TASK-SPLIT-02	Move M1, M2, M3, M7, M8, M9, M10, M15, M18, M19-M22 into foundation/src/	2h	REQ-SPLIT-04
TASK-SPLIT-03	Create utils/ crate with Cargo.toml (foundation dep, feature gates)	1h	REQ-SPLIT-02, SPLIT-03
TASK-SPLIT-04	Move M4, M5, M6, M11, M12, M13, M14, M16, M17 into utils/src/	2h	REQ-SPLIT-05
TASK-SPLIT-05	Add `#[cfg(feature = "...")]` gates to all utils modules	1.5h	REQ-SPLIT-03
TASK-SPLIT-06	Update all 5 system Cargo.toml with correct feature selections	1h	REQ-SPLIT-05
TASK-SPLIT-07	CI job: `cargo build -p tejas-foundation` standalone	0.5h	REQ-SPLIT-07
TASK-SPLIT-08	CI job: build utils/ with each feature individually	1h	REQ-SPLIT-08
TASK-SPLIT-09	Verify: foundation/ does NOT import nalgebra anywhere	0.5h	REQ-SPLIT-01
TASK-SPLIT-10	Verify: no circular dependency (foundation ↛ utils)	0.5h	REQ-SPLIT-06
Subtotal		**11h**	

Updated Crate Structure

```
tejas-intelligence/
├── foundation/                      # tejas-foundation crate
│   ├── Cargo.toml                   # sha2, ed25519-dalek, libm, serde ONLY
│   ├── .cargo/config.toml           # M18: FMA-disabled compiler flags
│   ├── clippy.toml                  # M18: Banned types
│   ├── ci/check_no_blas.sh          # M18: CI guard
│   └── src/
│       ├── lib.rs                   # Re-exports
│       ├── context.rs               # M1:  DeterministicContext
│       ├── primitives.rs            # M2:  Hash256, Ed25519*, LogicalTimestamp, Confidence
│       ├── id.rs                    # M22: DeterministicId<T>, all tags, all type aliases
│       ├── transport.rs             # M19: Envelope, Response, TransportError, CallMode
│       ├── router.rs                # M20: Transport trait, InProcessRouter, AuditSink
│       ├── client.rs                # M21: SiddhantaClient, DhruvaClient, etc.
│       ├── traits.rs                # M7:  Cross-system traits (5 traits)
│       ├── types/
│       │   └── cross_system.rs      # M3:  ExecutionGraph, StateFingerprint, etc.
│       └── helpers/
│           ├── hash.rs              # M8:  SHA-256, Merkle
│           ├── serialize.rs         # M9:  Canonical JSON, deterministic binary
│           ├── time.rs              # M10: Logical time helpers
│           └── audit.rs             # M15: Audit record builders
│
├── utils/                           # tejas-utils crate
│   ├── Cargo.toml                   # depends on foundation, feature-gated
│   └── src/
│       ├── lib.rs                   # Feature-gated re-exports
│       ├── types/
│       │   ├── perception.rs        # M4:  #[cfg(feature = "perception")]
│       │   ├── reasoning.rs         # M5:  #[cfg(feature = "reasoning")]
│       │   └── infrastructure.rs    # M6:  #[cfg(feature = "infra")]
│       └── helpers/
│           ├── math.rs              # M11: #[cfg(feature = "math")]
│           ├── linalg.rs            # M12: #[cfg(feature = "linalg")]
│           ├── validate.rs          # M13: #[cfg(feature = "validate")]
│           ├── determinism.rs       # M14: #[cfg(feature = "determinism")]
│           ├── id.rs                # M16: #[cfg(feature = "id-display")]
│           └── graph.rs             # M17: #[cfg(feature = "graph")]
│
├── praman/                          # depends: foundation + utils[full]
├── tejas/                           # depends: foundation + utils[graph, validate]
├── dhruva/                          # depends: foundation + utils[validate, infra]
├── siddhanta/                       # depends: foundation + utils[validate]
└── dristi/                          # depends: foundation + utils[math, validate, graph]
```

Updated Build Order

```
Sprint 0 (Week 1): Foundation Core — 42h
```

```
  M18: Compiler & Build Config            4.5h   ← FIRST
  M2:  Primitive Types                   12.5h
  M22: Generic ID Framework              16.0h   ← NEW (replaces old M16 ID helpers)
  M1:  DeterministicContext (updated)     9.0h   ← was 16h, reduced since IDs moved to M22

Sprint 1 (Week 2): Foundation Infra — 46h
  M8:  Hash Helpers                      10.5h
  M9:  Serialization Helpers             12.0h
  M10: Time Helpers                       2.5h
  M15: Audit Helpers                      3.0h
  M7:  Cross-System Traits                4.5h
  M3:  Compound Types (Cross-System)      4.5h   ← uses DeterministicId now
  SPLIT: Create foundation/ crate         9.0h   ← TASK-SPLIT-01..10

Sprint 2 (Week 3): Transport Layer — 36h          ← NEW
  M19: Transport Types                    7.5h
  M20: Transport Router                  14.0h
  M21: Typed Cross-System Client         14.5h

Sprint 3 (Week 4): Utils Crate — 48h
  M4:  Compound Types Perception          3.5h
  M5:  Compound Types Reasoning           5.5h
  M6:  Compound Types Infrastructure      4.0h
  M11: Math Helpers                       8.5h
  M12: Linear Algebra Helpers            14.0h
  M13: Validation Helpers                 4.5h
  M14: Determinism Helpers                4.0h
  M17: Graph Helpers                      6.5h
  (M16 ID Helpers → absorbed into M22)
```

Updated Totals

v3.0 Original (from Foundation_Utils_REQ_DES_TASKS_v3_0.md)

	REQs	Tasks	Hours
v3.0 Original	133	134	122.5h

v3.1 Addendum (this document)

Part	REQs	Tasks	Hours
A: Transport Layer (M19-M21)	21	28	36h
B: Generic ID System (M22 + M1 updates)	12	18	19h
C: Foundation/Utils Split	8	10	11h

Total Addendum	**41**	**56**	**66h**

Combined v3.1

	REQs	Tasks	Hours
Foundation crate (M1-M3, M7-M10, M15, M18-M22)	117	120	124h
Utils crate (M4-M6, M11-M14, M17)	49	60	53.5h
Crate separation tasks	8	10	11h
Grand Total v3.1	**174**	**190**	**188.5h**

Dev time: ~23.5 dev-days → ~4.7 weeks for 1 developer

Dependency Map (v3.1)

```
M18 (Compiler Config)
  ↓
M2 (Primitives)
  ↓
M22 (Generic ID Framework)     ← NEW
  ↓
M1 (DeterministicContext) ← updated with next_id<T>()
  ↓
M8 (Hash) → M9 (Serialize) → M10 (Time) → M15 (Audit)
  ↓
M7 (Traits) → M3 (Cross-System Types)
  ↓
SPLIT: Create foundation/ crate
  ↓
M19 (Transport Types) → M20 (Router) → M21 (Clients)     ← NEW
  ↓
[foundation/ crate COMPLETE]
  ↓
M4-M6 (Domain Types) → M11-M14, M17 (Helpers)
  ↓
[utils/ crate COMPLETE]
  ↓
[ALL SYSTEM CRATES CAN BEGIN]
```

---

v3.0 → v3.1 Migration Checklist

Item	v3.0	v3.1	Status
TraceId	`[u8; 16]` newtype	`DeterministicId<TraceTag>`	UPGRADED
SnapshotId	not defined	`DeterministicId<SnapshotTag>`	NEW
LedgerEntryId	not defined	`DeterministicId<LedgerTag>`	NEW
CertificateId	not defined	`DeterministicId<CertificateTag>`	NEW
ArtifactId	not defined	`DeterministicId<ArtifactTag>`	NEW
FrameId	`u64` newtype	`DeterministicId<FrameTag>`	UPGRADED
NodeId	`u32`	`DeterministicId<NodeTag>`	UPGRADED
RuleId	`u32`	`DeterministicId<RuleTag>`	UPGRADED
SessionId	not defined	`DeterministicId<SessionTag>`	NEW
Cross-system calls	Direct trait call	Envelope → Router → Handler	NEW
shared/ crate	single crate	foundation/ + utils/	SPLIT
M16 (ID Helpers)	trace_id_to_hex only	Absorbed into M22	MERGED
DeterministicContext.trace_counter	u64	id_counter: u64 (renamed, generic)	RENAMED

This addendum is applied ON TOP of Foundation_Utils_REQ_DES_TASKS_v3_0.md. All v3.0 module specs remain valid unless explicitly superseded here.